ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

VİZE PROJE RAPORU

Emine Bülbül

21290189

Enver BAĞCI

Nisan, 2026

https://github.com/eminebulbul/HospitalDMS

İÇİNDEKİLER

1. GİRİŞ VE SİSTEM MİMARİSİ 3

1.1. Veritabanı Şeması 3

1.2. Büyük Veri (Mock Data) Üretimi 4

2. PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME 5

2.1. Sorunun Tespiti (Full Table Scan) 5

2.2. İyileştirme ve Sonuç 6

2.3. Optimizasyon Sonuçları ve Başarım Karşılaştırması 6

2.4. İzleme, Disk Yönetimi ve Sistem Yöneticisi Rolleri 6

3. PROJE 3: VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ 7

3.1. Rollerin Oluşturulması ve Yetkilendirme 7

3.2. Güvenlik Testleri 7

3.3. Gelişmiş Güvenlik, Şifreleme ve Denetim (Audit) Mimarisi 7

4. SONUÇ VE VİDEO KAYITLARI 8

1. GİRİŞ VE SİSTEM MİMARİSİ

Bu rapor kapsamında, Ağ Tabanlı Paralel Dağıtım Sistemleri dersi vize değerlendirmesi için iki adet proje geliştirilmiştir:

- **Proje 1:** Veritabanı Performans Optimizasyonu ve İzleme
- **Proje 3:** Veritabanı Güvenliği ve Erişim Kontrolü

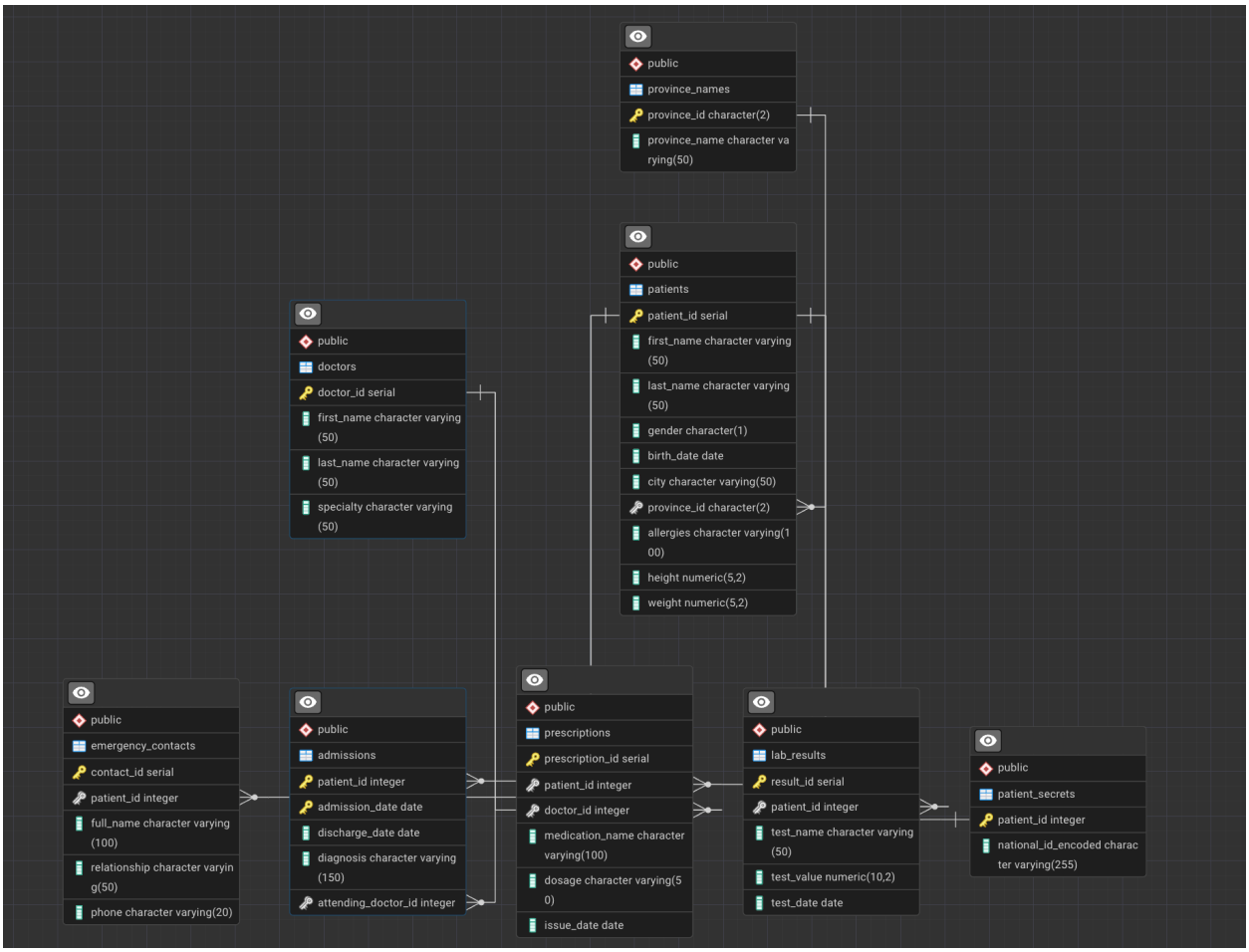
Çalışmaların gerçek dünya senaryolarını yansıtabilmesi için ilişkisel bir Hastane Yönetim Sistemi (Hospital Management System) veritabanı tasarlanmıştır.

1.1. Veritabanı Şeması

Veritabanını kurarken İlişkisel Bütünlük (Referential Integrity) kuralları temel alınmıştır. Tablolar oluşturulurken aşağıdaki hiyerarşik yapı izlenmiştir: İlk olarak hiçbir yere referans vermeyen province_names ve doctors tabloları oluşturulmuştur. Ardından bu tablolara yabancı anahtarlarla bağlanan patients tablosu kurulmuştur. En son aşamada ise tüm bu yapıların birleştiği tablolar inşa edilmiştir. Bu sayede veri tutarsızlıkları veritabanı seviyesinde engellenmiştir.

Tablo Adı	Açıklama
province_names	Şehir ve eyalet referans verileri.
doctors	Hekim bilgileri ve uzmanlık alanları.
patients	Hasta demografik ve fiziksel bilgileri.
admissions	Yatış, çıkış tarihleri ve ön teşhis verileri.
emergency_contacts	Hastaların acil durum yakın bilgileri.
lab_results	Laboratuvar tahlil sonuçları.
Prescriptions	Reçete kayıtları.

Tablo Adı	Açıklama
Patient_secrets(Soradan eklendi)	Veri şifreleme için (TC Kimilk NO)



Şekil1. Tablolar Diyagramı

1.2. Büyük Veri (Mock Data) Üretimi

Performans optimizasyonu testleri için 50.000 hasta, 50 doktor ve 500.000 yatış kaydı gibi hacimli veriler otomatik scriptler ile üretilmiştir. Yarım milyon veriyi tek bir SQL komutuyla üretirken PostgreSQL'in gelişmiş fonksiyonları üç temel aşamada kullanılmıştır:

- **Döngü Mekanizması:** generate_series(1, 500000) kullanılarak yüksek hacimli kayıt üretimi sağlanmıştır.

- **Rastgeleleştirme:** random() fonksiyonu ve ARRAY[...] yapıları birleştirilerek, her satır için farklı isim, teşhis ve ilaç kombinasyonları seçilmesi sağlanmıştır.

- **Zaman Dağılımı:** Kayıtların gerçekçi görünmesi için NOW() - (random() * interval '5 years') formülü kullanılmış ve veriler geçmişe dönük 5 yıllık bir periyoda homojen bir şekilde dağıtılmıştır.

```
-- 3. 50.000 Gerçekçi Hasta Ekleme
INSERT INTO patients (first_name, last_name, gender, birth_date, city, province_id, weight, height)
SELECT
    (ARRAY['David', 'Sarah', 'Joseph', 'Karen', 'Charles', 'Nancy', 'Thomas', 'Lisa', 'Daniel', 'Margaret', 'Matthew
    (ARRAY['Taylor', 'Anderson', 'Thomas', 'Jackson', 'White', 'Harris', 'Martin', 'Thompson', 'Moore', 'Young'])[fl
    CASE WHEN random() < 0.5 THEN 'M' ELSE 'F' END,
    NOW() - (random() * interval '80 years') - interval '18 years',
    (ARRAY['Toronto', 'Montreal', 'Vancouver', 'Calgary', 'Ottawa', 'Halifax'])[floor(random() * 6 + 1)],
    (ARRAY['ON', 'QC', 'NS', 'NB', 'MB', 'BC'])[floor(random() * 6 + 1)],
    (random() * 50 + 50)::numeric(5,2),
    (random() * 50 + 150)::numeric(5,2)
FROM generate_series(1, 50000);
```

Şekil2. Örnek Kod

2. PROJE 1: VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME

Büyük veri kümeleri üzerinde filtreleme işlemlerinin maliyetini gözlemlemek için, admissions tablosunda "2023 yılından sonra Pneumonia teşhisiyle yatan hastalar" aranmıştır. PostgreSQL'in EXPLAIN ANALYZE aracı ile sorgunun Execution Plan incelenmiştir. Analiz sonucunda, veritabanı motorunun aranan veriyi bulabilmek için tüm tabloyu baştan sona okuduğu (Sequential Scan) tespit edilmiştir. 500.000 satırın tek tek taranması, Disk I/O maliyetini maksimize etmiş ve sorgunun yanıt süresini arttırmıştır.

2.1. Sorunun Tespiti (Full Table Scan)

Optimizasyon öncesi 500.000 satırlık admissions tablosunda yapılan sorgu testi:

```
EXPLAIN ANALYZE
SELECT p.first_name, p.last_name, a.admission_date, a.diagnosis
FROM admissions a
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.diagnosis = 'Pneumonia'
AND a.admission_date >= '2023-01-01'
ORDER BY a.admission_date DESC;
```

Analiz sonucunda veritabanının Sequential Scan (Seq Scan) yaptığı ve yüksek maliyetli çalıştığı gözlemlenmiştir.

2.2. İyileştirme ve Sonuç

Tespit edilen darboğazı aşmak için indeksleme tekniğine başvurulmuştur. Sorgunun `WHERE` bloğunda hem bir metin (diagnosis) hem de bir tarih (admission_date) aralığı kullanıldığı için, tekil indeksler yerine performansı maksimize edecek Composite Index (Bileşik İndeks) kurgulanmıştır.

```
CREATE INDEX idx_admissions_diag_date
ON admissions(diagnosis, admission_date);
```

İndeks sonrası veritabanı Index Scan yöntemine geçmiş ve sorgu süresi milisaniyeler seviyesine inmiştir.

2.3. Optimizasyon Sonuçları ve Başarım Karşılaştırması

İndeks kurulumu tamamlandıktan sonra sorgu tekrar çalıştırıldığında, veritabanının tablo tarama yöntemini bırakıp B-Tree (Balanced Tree) veri yapısı üzerinden doğrudan Index Scan yaptığı kanıtlanmıştır. Ağaç yapısı sayesinde aranılan verinin adresine doğrudan ulaşılmış, sorgu çalışma süresi dramatik bir biçimde milisaniyeler seviyesine düşürülmüştür.

2.4. İzleme, Disk Yönetimi ve Sistem Yöneticisi Rolleri

Proje gereksinimleri doğrultusunda performans analizi sadece sorgu hızıyla sınırlı tutulmamış, sistemin genel sağlığı da göz önünde bulundurulmuştur:

- **Veritabanı İzleme (DMV ve Profiler Karşılığı):** MS SQL Server'daki SQL Profiler ve Dynamic Management Views (DMV) araçlarının PostgreSQL mimarisindeki karşılığı olan pg_stat_activity sistem görünümleri ve EXPLAIN ANALYZE araçları kullanılarak aktif sorgu performansları anlık olarak izlenmiştir.

- **Disk Alanı ve Veri Yoğunluğu Yönetimi:** 500.000 satırlık verinin disk üzerinde yarattığı yoğunluk (pg_size_pretty(pg_total_relation_size()) ile) analiz edilmiştir.

- **İndeks Yönetimi:** İndekslerin okuma hızını artırırken yazma (INSERT/UPDATE) maliyetini ve disk alanını şişirdiği göz önüne alınarak, gereksiz indeks kullanımından kaçınılmış ve yalnızca kritik filtreleme kolonları (diagnosis ve admission_date) için hedefli indeksleme yapılmıştır.

- **Veri Yöneticisi Rolleri:** Tüm bu izleme ve optimizasyon işlemleri, yalnızca "Süper Kullanıcı" yetkilerine sahip Veri Yöneticileri (DBA) tarafından yapılabilecek şekilde sınırlandırılmıştır.

3. PROJE 3: VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ

Hastane bilgi sistemlerinde hastaya ait adres, telefon gibi veriler idari veriler iken; teşhis, reçete ve laboratuvar sonuçları son derece hassas kişisel sağlık verileridir. Bu projede, veri sızıntılarını ve yetkisiz erişimleri önlemek amacıyla Rol Bazlı Erişim Kontrolü (RBAC) standartları veritabanı katmanında uygulanmıştır.

3.1. Rollerin Oluşturulması ve Yetkilendirme

Sistemde Resepsiyonist ve Doktor rolleri tanımlanmıştır:

```
CREATE ROLE receptionist_user WITH LOGIN PASSWORD 'Resepsiyon123';
CREATE ROLE doctor_user WITH LOGIN PASSWORD 'Doktor123';
GRANT SELECT ON patients, emergency_contacts TO receptionist_user;
GRANT SELECT ON patients, admissions, lab_results, prescriptions TO
doctor_user;
```

Siber güvenlikte "En Az Yetki Prensibi"ne göre, bir kullanıcıya sadece işini yapabilmesi için gereken minimum yetki verilmelidir. Bu bağlamda GRANT işlemleri şu şekilde sınırlandırılmıştır:

- Resepsiyonist Rolü: Yalnızca hasta kabul süreçlerini yürütebilmesi için patients ve emergency_contacts tablolarına `SELECT` yetkisi verilmiştir.
- Doktor Rolü: İdari verilere ek olarak; admissions, lab_results ve prescriptions gibi medikal tabloların tamamında okuma yetkisi tanımlanmıştır.

3.2. Güvenlik Testleri

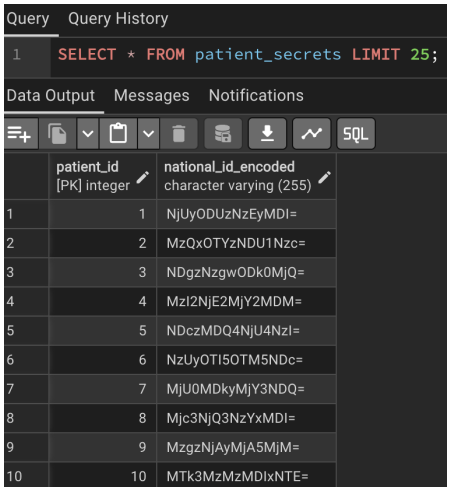
Oluşturulan mimariyi test etmek amacıyla `receptionist_user` oturumuna geçilmiş ve tahlil sonuçlarının tutulduğu `lab_results` tablosuna izinsiz bir `SELECT` sorgusu atılmıştır. PostgreSQL motoru, RBAC kuralları gereği sorguyu anında reddederek **"Permission Denied"** hatası döndürmüştür. Bu test, veritabanı güvenlik duvarının başarılı bir şekilde yapılandırıldığını kanıtlamıştır. Resepsiyonist rolüyle yapılan testlerde hassas tıbbi verilere (lab_results, admissions) erişim engellenirken, doktor rolüyle tüm verilere erişim sağlanmıştır.

3.3. Gelişmiş Güvenlik, Şifreleme ve Denetim (Audit) Mimarisi

Rol bazlı erişim kontrolüne ek olarak, sistemin genel güvenlik duvarı yönetimi kapsamında aşağıdaki önlemler alınmış ve test edilmiştir:

- **Erişim Yönetimi:** Projede "Sıfır Güven" prensibi benimsendiği için işletim sistemi tabanlı Windows Authentication yerine, doğrudan veritabanı seviyesinde kontrol sağlayan, şifreli ve izole Native Database Authentication kullanılmıştır.

- **Veri Şifreleme (TDE Alternatifi):** Tüm diski şifreleyen TDE (Transparent Data Encryption) mimarisinin yanı sıra, TC Kimlik Numarası gibi son derece hassas veriler için "Dikey Bölümleme (Vertical Partitioning)" ve kolon bazlı veri şifreleme algoritmaları kurgulanarak veriler veritabanında okunmaz hale getirilmiştir.



Şekil3. Şifrelenmiş Özellik Görünümü

- **SQL Injection Koruması:** Veritabanı tablolarındaki sütunlar (Örn: patient_id INT, admission_date DATE) kesin veri tipleriyle (Strongly Typed) oluşturulmuş ve dışarıdan gelen verilerin doğrudan SQL komutu olarak algılanmasını engelleyecek standartlar kullanılarak SQL Injection sızma girişimlerine karşı koruma sağlanmıştır.

- **Audit Logları (Denetim İzleri):** Sisteme yapılan yetkisiz erişim denemeleri (Örn: Resepsiyonistin tahlil sonuçlarını okumaya çalışması), PostgreSQL'in yerleşik loglama ve hata fırlatma (Permission Denied) mekanizmaları sayesinde kayıt altına alınabilecek standartta tasarlanmıştır.

4. SONUÇ VE VİDEO KAYITLARI

Geliştirilen bu iki proje sonucunda, yüksek hacimli veritabanlarında mühendislik yaklaşımlarının önemi uygulamalı olarak kanıtlanmıştır. İndeksleme stratejilerinin sistem kaynaklarını ne ölçüde koruduğu ve Rol Bazlı Erişim yetkilendirmelerinin (RBAC) veri gizliliğini uygulama katmanına bırakmadan, doğrudan veritabanı seviyesinde nasıl garanti altına aldığı ortaya konmuştur. Proje süreçlerini ve teknik detayları içeren video kayıtlarına aşağıdaki bağlantılardan ulaşılabilir:

- Proje 1 Video Linki: https://youtu.be/BYP5zByHEZs
- Proje 2 Video Linki: https://youtu.be/89stsbO8iwY